

REU SYSTEM V2.0

*Plan de Trabajo Integral para la Estabilización, Adaptabilidad Multi-Nivel e
Implementación del Módulo de Inscripciones por Cobrar*

Proyecto: Red Estudiantil Universitaria (REU) — Adaptación Global Escolar y Financiera

Estado del Arte: Full-Stack (Node.js, Express, MariaDB, Vanilla JS)

Propósito: Funcionalidad al 100% y Robustez a Nivel de Producción

Fecha de Emisión: Junio de 2026

1. INTRODUCCIÓN Y PROPÓSITO DEL PLAN

El presente documento constituye el **Plan de Trabajo Completo e Integral** para la optimización, reestructuración y culminación al 100% de la plataforma web **Red Estudiantil Universitaria (REU System)**. Partiendo de un software robusto con más de 24,000 líneas de código backend y frontend estructuradas bajo arquitectura MVC y JavaScript Vanilla, se identifican las brechas tecnológicas necesarias para llevar la aplicación a un nivel comercial y operativo absoluto.

Este plan unifica las directrices estratégicas abordadas para la ****Adaptabilidad Multi-Nivel**** (primaria, secundaria y universidad) y el diseño funcional y arquitectónico del módulo inédito ****Inscripciones por Cobrar****. El fin último es mitigar riesgos críticos de seguridad, garantizar la persistencia e integridad de datos relacionales en MariaDB y ofrecer una experiencia de usuario fluida y blindada del lado del servidor.

2. ARQUITECTURA DE ADAPTABILIDAD MULTI-NIVEL

Para que el ecosistema REU System pueda implementarse de forma transparente en planteles de educación básica (primaria), media (liceos/bachilleratos) e institutos técnicos, sin alterar el núcleo del código fuente backend, se establece una capa de abstracción sobre la base de datos conceptual.

2.1. Abstracción del Modelo de Datos

- **Estructura Curricular** (``carreras`` y ``pensum``): Se añade el campo semántico `Tipo_Nivel` (ENUM: 'Primaria', 'Media_General', 'Tecnico', 'Pregrado', 'Posgrado'). Cuando el sistema opere en nivel escolar, la "Carrera" se instanciará como "Educación Básica General" y los "Trayectos/Trimestres" mutarán lógicamente a "Años Escolares" o "Grados" (Ej: 1er Año, 5to Grado) mediante un mapeo dinámico en las vistas.
- **Lógica de Asignación de Docentes**: En entornos universitarios, la relación `docente_asignatura` es n-dimensional y fragmentada por secciones. En educación primaria, se habilita la asignación de un docente como "Maestro de Aula", centralizando múltiples asignaturas bajo un mismo identificador de grupo (Aula Común).
- **Módulo de Representantes (Obligatoriedad Escolar)**: Se diseña la inclusión de la entidad `representantes`, vinculada a través de una tabla intermedia de cardinalidad **1:N** hacia `estudiantes`. Esto habilita un nuevo rol con permisos restringidos de lectura para boletas de calificaciones, reportes de inasistencias y estados de cuenta financieros.

3. FORTALECIMIENTO Y NORMALIZACIÓN DE LA BASE DE DATOS (MARIADB)

El análisis técnico detectó prácticas de desnormalización orientadas a almacenar nombres de texto junto a llaves foráneas para eludir sentencias `JOIN`. Esto introduce un riesgo crítico de inconsistencia de datos.

3.1. Re-Normalización e Indexación

Se prohíbe explícitamente la duplicidad de strings de identificación nominal en tablas transaccionales u operativas. Toda consulta de nombres completos deberá resolverse mediante `INNER JOIN` o `LEFT JOIN` aprovechando la indexación nativa por llaves primarias en la tabla `usuarios`. Se configuran índices compuestos en campos de alta concurrencia, tales como `(id_estudiante, id_periodo)`.

3.2. Implementación Obligatoria de Soft Delete (Borrado Lógico)

Ningún registro de estudiantes, notas, periodos o pagos puede ser eliminado físicamente mediante sentencias `DELETE`, debido a que destruye la trazabilidad histórica y rompe la integridad referencial.

Directriz de Desarrollo: Se añadirá la columna `activo TINYINT(1) DEFAULT 1` a todas las tablas del sistema. El controlador genérico `genericController.js` interceptará las peticiones de eliminación para ejecutar en su lugar un `UPDATE tabla SET activo = 0 WHERE id = :id`. Las consultas `SELECT` incluirán por defecto la cláusula `WHERE activo = 1`.

4. OPTIMIZACIÓN DEL BACKEND Y SEGURIDAD ESTRICTA

4.1. Ciclo de Vida del JWT y Silent Refresh

Mantener la expiración del token de acceso en 5 minutos es una práctica de seguridad excelente. No obstante, para evitar la interrupción de la experiencia de usuario, se dictamina el desarrollo del mecanismo de **Silent Refresh**:

1. Al iniciar sesión, el servidor emitirá un `accessToken` (JWT, 5 min de vigencia) transmitido en el cuerpo de la respuesta, y un `refreshToken` (JWT, 7 días de vigencia) almacenado en una cookie con las propiedades `HttpOnly`, `Secure` y `SameSite=Strict`.
2. El frontend, a través de un interceptor en el archivo global de peticiones, detectará el error HTTP 401 (Token Expirado).
3. De manera transparente en segundo plano, realizará una llamada al endpoint `POST /api/auth/refresh-token` para renovar el token de acceso y re-ejecutar la operación original del usuario sin sacarlo del sistema.

4.2. Desmitificación de la Seguridad en Frontend vs. Servidor

El bloqueo de la consola del navegador (tecla F12, clic derecho) incluido en versiones previas constituye una medida cosmética y vulnerable. Cualquier usuario con conocimientos básicos puede evadirlo inhabilitando JavaScript o interactuando de forma directa con la API mediante clientes como Postman o cURL.

Regla de Oro del Proyecto: La seguridad reside única y exclusivamente en el backend. El middleware `checkJWT.js` debe validar la autenticidad de la firma del token y contrastar el rol del usuario directamente contra la base de datos en cada petición protegida. Se incorporará la librería `joi` o `express-validator` en las capas de enrutamiento para realizar una sanitización estricta de datos entrantes.

5. DESARROLLO DEL MÓDULO: INSCRIPCIONES POR COBRAR

Este módulo inédito automatiza la conciliación financiera y el desbloqueo académico. Se integra de forma orgánica con la tabla existente `estudiante_periodo_academico`, extendiendo su alcance para actuar como libro auxiliar de cuentas por cobrar.

5.1. Ciclo de Vida de la Transacción

Fase	Acción del Sistema / Actor	Estado del Pago	Impacto Académico
1. Pre-Inscripción	El estudiante selecciona carga académica en el frontend. El sistema computa tasas según el nivel escolar o créditos universitarios.	Pendiente	Horario reservado temporalmente; inaccesible en actas oficiales.
2. Reporte / Pago	El estudiante carga la referencia bancaria, o el cajero administrativo procese efectivo/pago móvil directamente en taquilla.	Procesando	Bloqueo preventivo de la matrícula en espera de confirmación bancaria.
3. Conciliación	El rol Administrativo valida la acreditación de fondos y presiona "Aprobar Pago".	Aprobado / Solvente	Liberación Total: Inclusión formal en listas, horarios y habilitación para calificaciones.

5.2. Especificación de Endpoints Backend

GET `/api/administrativo/inscripciones-pendientes`

- **Descripción:** Recupera la lista de cuentas por cobrar pendientes de validación.
- **Query SQL interna:**

```
SELECT ep.id, u.cedula, u.nombre, u.apellido, ep.monto_total, ep.estado_pago FROM
estudiante_periodo_academico ep INNER JOIN estudiantes e ON ep.id_estudiante = e.id
INNER JOIN usuarios u ON e.id_usuario = u.id WHERE ep.estado_pago IN ('Pendiente',
'Procesando') AND ep.activo = 1;
```

POST `/api/administrativo/registrar-pago/:id`

- **Payload Requerido:** `{ referencia: String, metodo_pago: String, monto_confirmado: Decimal }`.
- **Acciones del controlador:** Actualiza el estatus a `'Aprobado'`, inserta la auditoría en `bitacora.js` especificando el ID del administrador actuante y el monto verificado, y libera el estado de la matrícula.

5.3. Arquitectura Frontend (UI/UX Vanilla)

Siguiendo los estándares del proyecto (vistas desacopladas e inyectadas dinámicamente mediante `shared.js`), se estructuran dos componentes clave:

- **Vista de Tabla Dinámica:** Consumo del endpoint de pendientes empleando las utilidades existentes de `genericTable.js` and `paginacion.js`. Se prohíbe el uso de frameworks pesados, garantizando tiempos de renderizado instantáneos.
- **Formulario Modal de Conciliación:** Al hacer clic sobre una fila, se despliega una modal nativa. El campo "Método de Pago" se instancia mediante un objeto `Tom Select` optimizado. El envío del formulario despacha una petición asíncrona mediante `fetch`, destruye la fila de la vista con una animación sutil y emite una alerta no intrusiva de éxito.

5.4. Automatización: Generación de Recibo de Pago Digital

Una vez aprobada la transacción con éxito, el frontend invocará una rutina automatizada basada en la librería **jsPDF** (inyectada por CDN en el proyecto). Esta función creará un documento en formato PDF descargable e imprimible con las siguientes especificaciones institucionales:

- Membrete corporativo y logotipo oficial de la Red Estudiantil Universitaria (REU) en formato vectorial SVG/Base64.
- Número correlativo único de recibo financiero y marcas temporales de auditoría del sistema.
- Bloque de datos de identidad del alumno (Cédula, Nombres, Apellidos, Carnet único corporativo).
- Concepto formal del cobro (Ejemplo: *"Inscripción Regular - Período Académico 2026-1"*).
- Un distintivo visual de gran formato en color verde esmeralda con la leyenda: **"PROCESADO / ESTADO: SOLVENTE"**.

6. CRONOGRAMA DE TRABAJO Y FASES DE IMPLEMENTACIÓN

Para garantizar la ejecución coordinada de estas mejoras sin romper las 24,000 líneas de código base, el plan se divide en cuatro fases consecutivas en un horizonte temporal sugerido de 6 a 8 semanas:

Fase	Semanas	Entregables Principales	Criterio de Aceptación
Fase I: Persistencia y Core	Semana 1 - 2	Refactorización de <code>genericController.js</code> para borrado lógico (Soft Delete). Migración y eliminación de duplicados nominales en MariaDB.	Pruebas de integridad referencial superadas sin pérdida de historial.
Fase II: Blindaje de Seguridad	Semana 3	Implementación de cookies seguras <code>HttpOnly</code> para Refresh Tokens. Despliegue de validadores de esquemas del lado del servidor.	Expiración simulada de JWT se renueva de forma transparente sin desloguear.
Fase III: Módulo Financiero	Semana 4 - 5	Desarrollo de vistas HTML/JS para "Inscripciones por Cobrar". Programación de endpoints administrativos y disparadores de bitácora.	Inserción correcta del estado 'Pendiente' tras simular una pre-inscripción.
Fase IV: Cierre y Reportes	Semana 6	Maquetación e integración del motor jsPDF para recibos de pago. Pruebas de carga con volumen de datos simulado mediante <code>regenerar-sql.js</code> .	Emisión de PDFs correctos con membrete y estatus visual "Solvente".